Session Three

Beyond Context - OpenCode

Understanding the limits and discovering alternatives

Before We Start

🤝

This is a learning session, not a sales pitch

📚 Learning Together

- Not an AI expert, just exploring
- Figuring this out as a team

💬 Open Discussion

- Questions encouraged
- Share your experiences

Goal: Understand & make informed choices

Recap: Our Journey So Far

🎯 Session One

From Chatbot to Agent

- AI without context vs. with context
- Demo: Same prompt, different results
- Context-aware tools
- Direct file modification

🎨 Session Two

Prompt Engineering

- The 5 Pillars framework
- Iterative refinement
- Common anti-patterns
- When NOT to use AI
- Sharing prompts with the team

Today's Journey

Part 1: Understanding the Limits

15 min

Context windows and 6 management strategies

Part 2: Where We Are Today

5 min

GitHub Copilot: What works, what's missing

Part 3: OpenCode Alternative

10 min

What it is, how it differs, when to use it

Part 4: Discussion & Takeaways

10 min

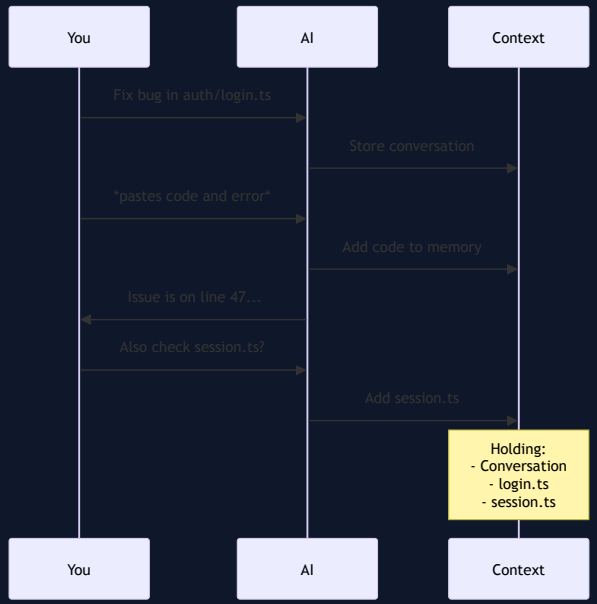
Key lessons and Q&A

🧠 Part 1

Understanding the Limits

The Context Window

Think of it as AI's working memory



Context Window Sizes (2026)

Impressive, right? But here's the catch...

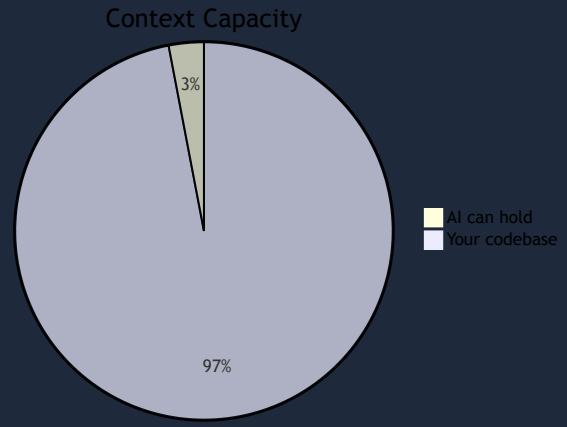
The Real Challenge

Your Project

```
src/
├── components/ (247 files)
├── services/ (89 files)
├── utils/ (156 files)
├── api/ (201 files)
└── tests/ (892 files)
```

5000+ files

AI's Memory



~150 files (3%)

Managing Context: 6 Best Practices

1. Configuration Files

Tell AI about your project upfront

Create a `CLAUDE.md` or `.cursorrules` file:

```
# Project: TF1 ITPV Platform

## Tech Stack
- React 18 with TypeScript
- Styling: Tailwind CSS + shadcn/ui components
- State: Redux Toolkit
- API: REST with Axios
- Testing: Jest + React Testing Library

## Conventions
- Component naming: PascalCase (e.g., UserProfile.tsx)
- File structure: src/features/[feature-name]/
- Imports: Use absolute imports via @/ alias
```

Why it works: AI reads this ONCE, applies it to ALL responses

2. Code Maps (Summaries)

Use high-level summaries instead of full files

```
❌ BAD: Load all 50 component files into context

✅ GOOD: Create a code map
```

Example code map:

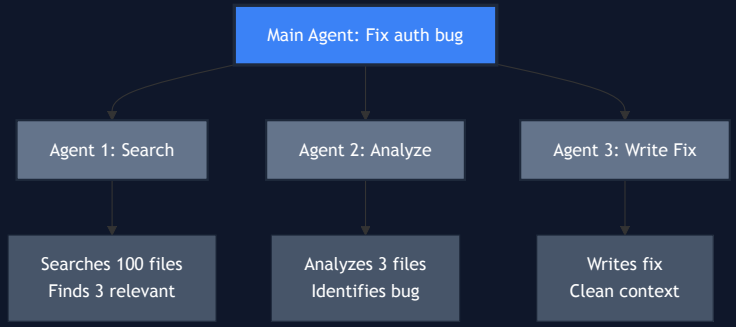
```
# Frontend Architecture

## Authentication Flow
- src/features/auth/LoginForm.tsx - Login UI
- src/features/auth/AuthProvider.tsx - Auth state management
- src/services/auth.api.ts - API calls to /api/auth/*

## User Management
- src/features/users/UserList.tsx - Display users
- src/features/users/UserProfile.tsx - User details
- src/services/users.api.ts - API calls to /api/users/*
```

Result: AI understands the structure without loading every file

3. Subagents (Parallel Workers)

🤖

Delegate tasks to specialized workers



Each agent has its own context

4. Dynamic Scoping

Let AI find what it needs

```
❌ BAD:
"Here are 20 files I think might be relevant.
 Can you find the bug?"

✅ GOOD:
"Find where user sessions are managed and fix
 the early expiration bug"
```

Why it works:

- AI uses search/grep to find relevant files
- Only loads what it actually needs
- Your context stays clean

Modern tools (Copilot, OpenCode) can explore the workspace autonomously

5. Chunking Large Tasks

Break big tasks into focused sessions

```
❌ BAD (one massive task):
"Refactor the entire authentication system, add OAuth,
 update all tests, and improve error handling"

✅ GOOD (4 separate sessions):
Session 1: "Audit current auth code and create refactor plan"
Session 2: "Add OAuth support (code only, no tests yet)"
Session 3: "Write tests for OAuth integration"
Session 4: "Improve error messages in auth flows"
```

Each session = focused context = better results

6. Session Continuity

Give AI "external memory"

Some tools support persistent sessions:

- **OpenCode:** Workspaces survive restarts
- **Cursor:** Session history (limited)

For tools without it, create handoff notes:

```
# Session Summary

## What We Did
- Refactored AuthProvider to use React Context
- Moved API calls to auth.api.ts

## Next Steps
- [ ] Add error handling for failed login
- [ ] Write tests for AuthProvider

## Files to Load
- src/features/auth/AuthProvider.tsx
```

Context Management Summary

Strategy	What It Does	When to Use
Config files	Project-wide instructions	Always (set it and forget it)
Code maps	High-level architecture view	Large codebases, onboarding
Subagents	Parallel workers, isolated context	Complex multi-step tasks
Dynamic scoping	Let AI search & find files	You don’t know exact file locations
Chunking	Break big tasks into sessions	Refactoring, large features
Session continuity	Remember across sessions	Multi-day projects

Golden rule: Less is more. Focused context beats overloaded context.

🔍 Part 2

Where We Are Today

GitHub Copilot in 2026

✅

WHAT COPILOT DOES WELL

- Excellent inline suggestions
- Context-aware with @workspace
- Direct file editing in VS Code
- Subagents support (since Oct 2025)
- Custom agents via .github/agents
- Workspace exploration with grep/search

⚠️

WHAT'S MISSING FOR TF1

- ❌ **No MCP support** (our plan doesn't allow it)
- ❌ **No nested agents** (flat orchestration only)
- Vendor lock-in (GitHub/Microsoft ecosystem)
- Limited model choice (GitHub's selection)

Key limitations: No MCP + No nested agents = limited extensibility

⚡ Part 3

OpenCode - The Alternative

What is OpenCode?

Open-source AI coding agent

Created by: SST team
GitHub: ⭐ 93,000+ stars
License: MIT (fully open)

First release: Dec 2024
Philosophy: You own your tools

✨ Use any AI model you want

🔒 No vendor lock-in

🤝 Community-driven development

💾 Full control over your data

OpenCode vs GitHub Copilot

Feature	Copilot	OpenCode
Inline completion	✅	❌
Workspace search	✅	✅
Subagents	✅	✅
🎯 **NESTED AGENTS**	❌	✅
🎯 **MCP SUPPORT**	❌*	✅
Model Choice	~5 models	75+ providers
Open Source	❌	✅
Local Models	❌	✅ (Ollama)

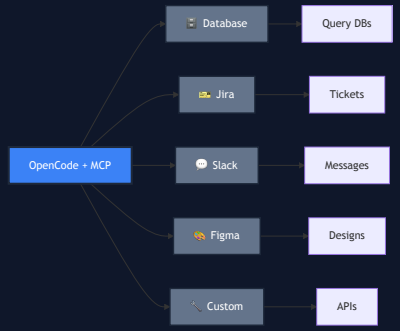
*MCP requires GitHub Copilot Extensions plan (not in our plan)

🔌 What is MCP? (Preview)

Model Context Protocol = "Plugins for AI"

Standard protocol for AI tool integration

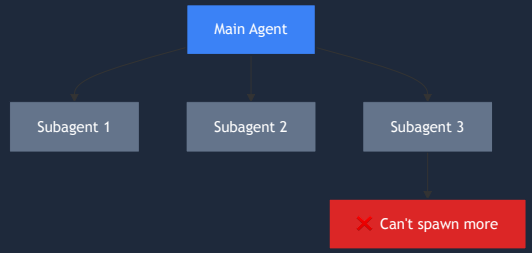
Created by: Anthropic **Released:** Nov 2024 **Purpose:** Connect AI to tools



The Nested Agents Advantage

GitHub Copilot

Flat orchestration



Subagents can't spawn sub-subagents

OpenCode

Nested orchestration

True hierarchical delegation

🎭 Multi-Provider Advantage

Use ANY AI model

🤖

OpenAI

GPT-4o, GPT-4, GPT-3.5

🧠

Anthropic

Claude 3.7 Sonnet, Opus

✨

Google

Gemini 2.5 Pro, Flash

☁️

AWS Bedrock

Claude via AWS

🔷

Azure OpenAI

Enterprise models

💻

Ollama

100% local, offline

75+ providers total

You decide: which model, where data goes, cost vs. performance

⚙️ How OpenCode Works

Key Features

🔧 **Workspace Tools**

- Direct file editing (read, write, search)
- Terminal access & Git integration

🤖 **Agent System**

- Subagents for parallel work
- Nested agents (unlimited depth)

🔌 **Extensibility**

- MCP support (connect to external tools)
- Custom slash commands

💾 **Persistence**

- Workspaces saved across sessions
- Resume work after restart

Available Interfaces

```
VS Code Extension
• Inline in your editor
• Most popular interface
```

```
CLI (Command Line)
• Terminal-based interaction
• Great for SSH/remote work
```

```
Web UI
• Browser-based (beta)
• No installation needed
```

Recommended: VS Code extension (best experience)

Share Sessions

Collaborate on AI conversations

```
YOU: "Fix the authentication bug"
OpenCode: *analyzes, proposes fix*

YOU: Share session link with teammate

TEAMMATE: Opens link, sees full conversation
TEAMMATE: "Also check session timeout logic"
OpenCode: *continues from where you left off*
```

Use cases:

- Code reviews with AI context
- Pair programming asynchronously
- Knowledge transfer

When to Use What?

GitHub Copilot

- Inline code completion
- Quick questions about open files
- Single-file refactoring
- Learning new APIs
- Simple workspace exploration

Best for: Day-to-day coding

OpenCode

- Multi-file refactoring (nested agents)
- Complex bug hunts across modules
- Architecture exploration
- MCP integrations (Jira, DBs, APIs)
- Experimenting with AI models

Best for: Complex, multi-step work

Use both - they complement each other

Resources

Official Documentation:

- OpenCode Website
- GitHub Repository (93k+ stars)
- Documentation
- Discord Community

Context Management:

- Anthropic: Best Practices
- Understanding Context Windows

Previous Sessions:

- Session One: From Chatbot to Agent
- Session Two: Prompt Engineering

💡 Part 4

Discussion & Takeaways

⚠️ A Word of Caution

From Dax Raad (OpenCode Creator)

"AI coding tools are extremely powerful, but they're not magic. The best results come from developers who understand:

1. What they're trying to build

2. How to verify the AI's output

3. When to step in and take control

Treat AI as a junior developer: skilled but needs guidance."

Key insight: You're still the architect. AI is the builder.

Key Takeaways

🧠

1. Context is finite

- Use config files (CLAUDE.md)
- Create code maps
- Leverage subagents

2. GitHub Copilot is excellent

- Inline completion unmatched
- Good for single-file work
- Limited: flat orchestration, no MCP

⚡

3. OpenCode excels at orchestration

- Nested agents for complex tasks
- MCP for Jira, databases, APIs
- Multi-provider flexibility

🤝

4. Use both tools

- Copilot for day-to-day
- OpenCode for complex work
- They’re complementary

Questions & Discussion

Let’s share experiences:

💭 Have you hit context limits with Copilot?

🔧 What complex tasks could benefit from nested agents or MCP?

🚀 Interest in a future MCP deep-dive session?

Thank You!

Let's keep learning together

📚

Resources

Check the session README for links

💬

Continue the Discussion

Share your thoughts in #iptv_dev_with_ai